

Predecir la Salud de la Batería de una Laptop con Machine Learning

Ximena Pérez Escalante

A01751827

Instituto Tecnológico y de Estudios Superiores de Monterrey

Agosto 2026

Resumen/Abstract ... *Keywords (palabras clave): Regresión lineal, ...*

Introducción

Una gran parte de la población trabaja con una laptop, ya sea para la escuela, el trabajo o uso personal. Y con cada laptop se vive el mismo problema, la degradación de la batería. De acuerdo al uso que se le dé, depende la rapidez con la que esto ocurra.

Por eso es común ver a personas que cuando usan su laptop, nunca la desconectan de la corriente. Y al preguntarles por qué hacen esto, su respuesta es que se le acaba muy rápido la batería, por lo que es más fácil dejarla conectada. Pero aunque eso ayuda en el momento, termina empeorando la salud de la misma.

Hoy en día, la mayoría de las laptops tienen baterías de iones de litio, cuyas

características son que se cargan rápidamente, se descargan a un ritmo constante y poseen una alta densidad energética que permite utilizar celdas pequeñas. De igual manera, la capacidad de las celdas de iones de litio se reduce después de un cierto número de veces en que la batería se carga y posteriormente descarga.

Existen múltiples artículos donde se describen maneras en las que se puede alentar el proceso de degradación de la batería. Entre ellas se encuentran:

- Evitar las descargas profundas frecuentes
- Mantener la batería dentro de un rango de aproximadamente el 20% y 80%

- No usar o cargar la laptop a altas temperaturas.
- Evitar almacenar la laptop completamente cargada o completamente descargada.

A pesar de que se documenten estas alternativas, comúnmente, los usuarios suelen tener malos hábitos de uso, por lo que se busca conocer cómo impactan directamente distintas variables, la salud de la batería para saber específicamente cómo preservar la salud de la misma.

A partir de esto, se planteó el objetivo de predecir la salud de la batería (battery health) medida en porcentaje, a partir de diferentes variables descritas en la siguiente sección.

Descripción del Dataset

Se utilizó “Laptop Battery Health Prediction Dataset”. Un Dataset sintético que simula el comportamiento de una laptop, modelando los cambios en la batería de la misma dependiendo del uso que se le dé. En total, tiene 4600 instancias y 10 features descritos en la siguiente tabla:

Nombre	Significado	Tipo de dato
Battery Age	Edad de la batería medida en años	Integer
Daily Usage Hours	Promedio de horas de uso al día	Decimal
Gaming User	Indica si el usuario juega videojuegos frecuentemente (Sí/No)	Integer
Design Capacity	Capacidad original de la batería cuando era nueva (mAh)	Integer
Cycle Count	Número total de ciclos charge-discharge completados	Integer
CPU Usage	Promedio de utilización del CPU durante uso diario (%)	Decimal
GPU Usage	Promedio de utilización del GPU durante uso diario (%)	Decimal
Power Consumption	Promedio de energía consumido durante la operación (Watts)	Decimal
Average Temperature	Promedio de temperatura de la batería en operación (°C)	Decimal

Full Charge Capacity	Máxima capacidad de carga actualmente después de degradation	Decimal
----------------------	--	---------

Tabla 1: Descripción de Features del Dataset

Al representar datos en un modelo de Machine Learning, tenemos dos tipos de datos:

- *Númericos*: Battery Age, Daily Usage Hours, Design Capacity, Cycle Count, CPU Usage, GPU Usage, Power Consumption, Average Temperature, Full Charge Capacity.
- *Catagóricos*: Gaming User.

A partir de estos tipos podemos realizar otra división en numérico discreto y continuo:

- *Númericos Discretos*: Battery Age, Design Capacity, Cycle Count.
- *Númericos Continuos*: Daily Usage Hours, CPU Usage, GPU Usage, Power Consumption, Average Temperature, Full Charge Capacity.
- *Númerico discreto de clase*: Gaming User.

ETL

ETL, por sus siglas en inglés se refiere al proceso: *Extract - Transform - Load*.

Extracción

Esta fase consiste en obtener los datos que se utilizarán en el modelo. Esto se puede realizar de distintas maneras. Para este trabajo, se extrajo la totalidad de los datos de un Dataset (full-extract), el cual es un archivo de tipo .csv, lo que quiere decir que está en un formato preparado para iniciar el proceso de transformación.

Transformación

Cuando se termina la fase de extracción, lo más probable es encontrar:

- Datos faltantes por columna o filas completas.
- Datos atípicos (outliers).
- Columnas irrelevantes que no proporcionan ningún valor al entendimiento de los datos.
- Filas duplicadas.

Al revisar el Dataset se identificó que no había datos nulos o filas duplicadas. Lo cual quiere decir que no es necesario eliminar filas o imputar valores.

Análisis Exploratorio de Datos (AED)

Análisis de Correlación entre variables

Continuando con el análisis exploratorio de los datos, se optó por calcular la matriz de correlación de las variables, para después hacer un mapa de calor. Ya que este permite visualizar esto de manera clara, donde el rojo significa correlación positiva y el azul una correlación negativa.

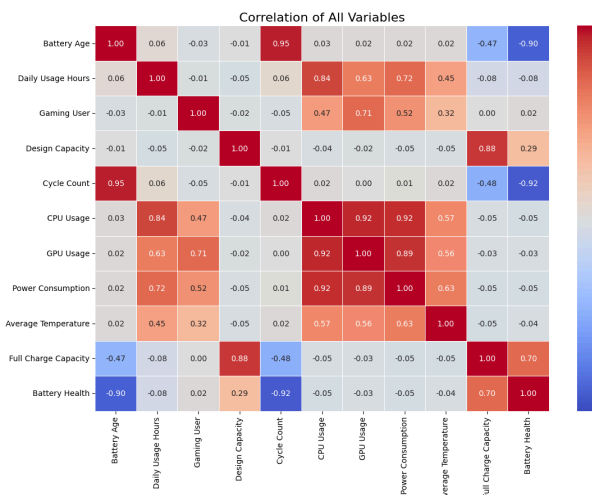


Figura 1: Mapa de Calor de Correlación entre variables

Correlaciones positivas con Battery Health (de mayor a menor):

- Full Charge Capacity (0.70)

- Design Capacity (0.29)

Correlaciones negativas con Battery Health (de mayor a menor):

- Cycle Count (0.92)
- Battery Age (0.90)

En este caso, es necesario poner atención a las relaciones, porque la fórmula para calcular la salud de la batería de una laptop es la siguiente:

$$Battery\ Health = \left(\frac{Full\ Charge\ Capacity}{Design\ Capacity} \right)$$

Por lo que al ver que estas dos variables son las que muestran la mayor relación positiva con Battery Health significa que al mantenerlas en el Dataset para entrenar el modelo, puede generar que el mismo sea inservible. Ya que conocemos esta fórmula y su relación directa con la variable que queremos predecir, es necesario eliminarlas del Dataset.

Valores atípicos (outliers)

Encontrar valores atípicos puede sesgar los datos, perjudicando las predicciones que genera el modelo. Sin embargo, en

otras ocasiones esto puede ser de utilidad para predecir datos reales.

Para identificar estos valores en el Dataset, se generaron Diagramas de caja y bigotes (Boxplots) por cada feature. A continuación se muestran únicamente las variables que cuentan con valores atípicos:

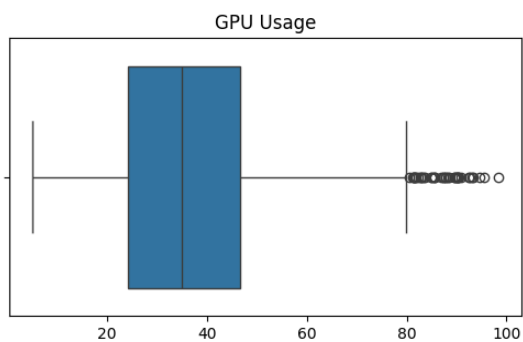


Figura 2: GPU Boxplot

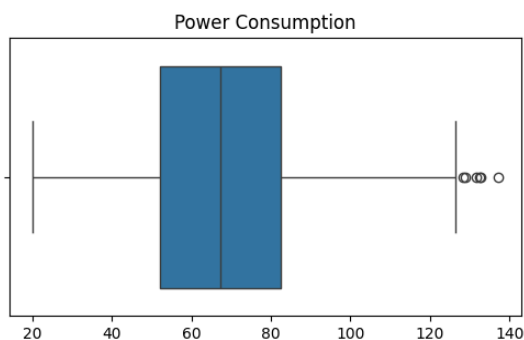


Figura 3: Power Consumption Boxplot

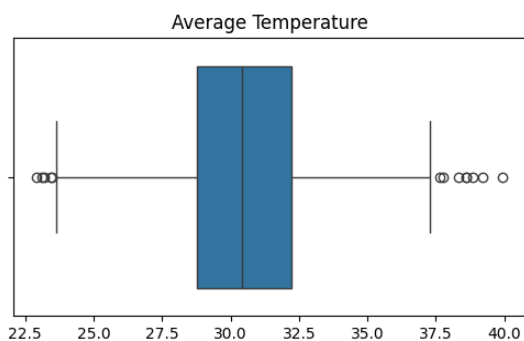


Figura 4: Average Temperature Boxplot

Debido al contexto del proyecto, donde se busca entender cómo cada variable impacta en la degradación de la batería de una laptop, se decidió no eliminar las instancias con valores atípicos de cada columna. Esto con el fin no sólo de mantener los datos realistas y preparar el modelo para estos casos, sino porque estos valores no son errores de mediciones, reiterando que es importante conservar dichos escenarios realistas.

Escalamiento de los datos

En este Dataset los features (excluyendo la variable categórica) tienen diferentes rangos numéricos, lo cual puede afectar de manera el modelo. Por ello, es necesario escalar las variables numéricas continuas.

División del Dataset en Entrenamiento y Prueba

Es importante mencionar que antes de realizar el escalamiento, se debe separar el Dataset en features (x) y target (y). Posteriormente se dividen ambos en entrenamiento (80%) y prueba (20%).

Esto con el objetivo de evitar el filtrado de datos.

Una vez realizado esto, se utilizó la estandarización como método de escalamiento, definido por la siguiente fórmula:

$$X_{\text{estandarizada}} = \frac{X - \text{promedio}(X)}{\text{std}(X)}$$

La razón por la que se eligió este método es que es comúnmente utilizado para regresiones lineales, por lo que se ajusta a la necesidad de este trabajo.

Carga de datos (Loading)

Para esta etapa simplemente se cargó el Dataset con las modificaciones que fueron realizadas anteriormente.

Marcando el final del proceso de ETL.

Modelo de Regresión Lineal v1.0

Al contar con un problema donde el objetivo es predecir una variable numérica, las predicciones se deben realizar con un modelo de regresión lineal, cuya fórmula es:

$$y = mx + b$$

Donde m (pendiente) y b (bias) son los parámetros.

Este modelo está conformado por tres funciones: hipótesis, costo y optimización. Las cuales se describirán a continuación.

Función de hipótesis

Es la función de regresión lineal inicial, la cual no ha sido comprobada. Para saber cuál es el mejor modelo, se comparan múltiples hipótesis.

Función de costo

Permite saber qué tan bien se ajusta el modelo a los datos. Es decir, compara la distancia entre la predicción del modelo y el valor real. En este caso se utilizará el Mean Squared Error, cuya fórmula es:

$$\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Función de optimización

Como su nombre lo indica, es la que optimiza el modelo para disminuir el costo, encontrando el valor ideal de m . Como entrada tiene la función de costo, que ya incluye la función de hipótesis.

Para realizar esto, se utiliza la gradiente descendiente, que calcula la derivada de la pendiente, lo que permite saber si el valor de m debe aumentar o disminuir para llegar a su valor ideal. Esto se expresa de la siguiente manera:

$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

Donde θ = parámetros a optimizar;
 α = tasa de aprendizaje; J = función de costo.

Resultados

Entrenamiento

Después de correr la función de optimización por 1000 epochs, se llegó a un costo total de aproximadamente 8670.178341781264.

La gráfica se ve de la siguiente manera:

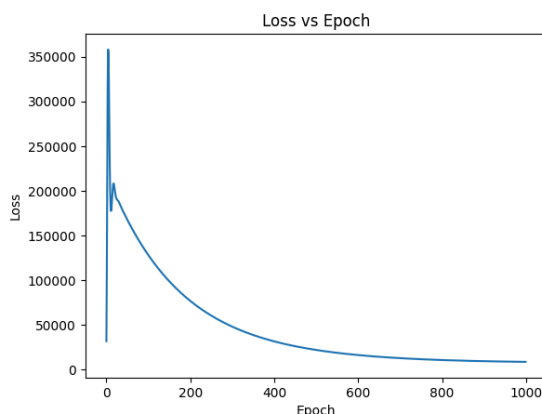


Figura 5: Error en train

Los parámetros quedaron de la siguiente manera:

- Θ_1 : -2.61347732e-01
- Θ_2 : 1.74268983e+00
- Θ_3 : -5.57213901e+00
- Θ_4 : 3.43738379e+02
- Θ_5 : 7.95004751e-04
- Θ_6 : -7.36261940e+00
- Θ_7 : -1.21912628e+01
- Θ_8 : -8.42952928e+00

Prueba

Usando los parámetros actualizados, se obtuvieron los siguientes resultados al realizar predicciones:

	Actual	Predicted
0	74.042368	355.156281
1	97.853490	103.721396
2	93.995585	50.749645
3	91.591248	88.403340
4	72.694413	5.360052
..	...	...
235	98.587142	90.181925
236	77.886805	66.996431
237	67.243703	47.632027
238	84.224161	36.862191
239	81.458552	89.532363

Figura 6: Predicciones en test

A simple vista, es claro que hay unas predicciones que se acercan al

resultado, pero hay muchas más que se alejan demasiado.

Graficando un scatter plot, es más fácil observar el desempeño del modelo:

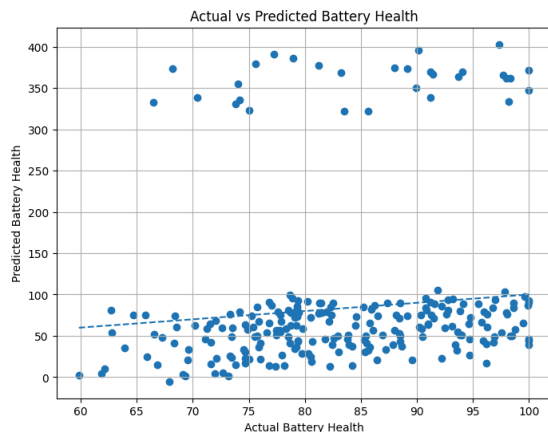


Figura 7: Scatterplot test predictions

La mayoría de las predicciones se encuentran cerca de la línea, sin embargo, lo ideal sería que esta pasara por en medio de ellos. Se puede observar que muchos están por debajo de ella, por lo que es algo que se debe considerar para realizar los ajustes necesarios.

¿Overfitting o Underfitting?

Al realizar predicciones con ambos train y test, se observa el mismo patrón. El modelo no se ajusta a los valores, ya que la mayoría de las predicciones quedan por debajo de la línea. Esto quiere decir que existe underfitting, ya

que el modelo tiene una baja complejidad y no se observó una diferencia significativa entre los resultados de train y test.

Siguientes pasos: Ajustar el modelo para tener la v2.0

Bibliografía

<https://www.kaggle.com/datasets/dharmendrapandit12/laptop-battery-health-dataset>

<https://support.microsoft.com/en-us/windows/experience/power-battery/caring-for-your-battery-in-windows>